

CUSTOMER CHURN PREDICTION

Using Machine Learning

Telco Customer Churn Dataset

Full Project Report

Teams: Keshav Gupta & Mudassir Iftikhar

1. Project Introduction

Customer churn — the phenomenon where customers stop doing business with a company — is one of the most critical challenges facing telecommunications companies. Retaining an existing customer is significantly less costly than acquiring a new one, making churn prediction an important business intelligence task.

This project builds a complete, production-style machine learning pipeline to predict customer churn from the publicly available Telco Customer Churn dataset, made available through IBM. The project follows a structured, modular Python architecture with separate source files for each stage of the ML workflow.

1.1 Project Objective

The objective of this project is to predict whether a customer will churn (leave the service) based on their demographic, behavioural, service usage, and billing features. The project compares multiple ML models, evaluates them with standard classification metrics, analyses feature importance, and derives business insights from the results.

1.2 Methodology Overview

- Data Loading and Inspection
- Data Preprocessing (type fixing, missing values, target encoding)
- Exploratory Data Analysis (EDA)
- Feature Engineering (encoding, scaling, SMOTE for class imbalance)
- Model Training (5 classifiers)
- Model Evaluation (Accuracy, Precision, Recall, F1, ROC-AUC)
- Hyperparameter Tuning (GridSearchCV)
- Feature Importance & SHAP Interpretation

1.3 Models Used

- Logistic Regression
- Decision Tree
- Random Forest
- XGBoost (Gradient Boosting)
- LightGBM (Gradient Boosting)

1.4 Project Structure

The project is organised as a fully modular Python package:

- `src/config.py` — All paths and constants
- `src/data_loader.py` — CSV loading and inspection
- `src/preprocessing.py` — Data cleaning pipeline
- `src/eda.py` — EDA plots generation
- `src/feature_engineering.py` — Encoding, scaling, SMOTE, train-test split
- `src/train_models.py` — Model training and saving
- `src/evaluate.py` — Metrics, confusion matrices, ROC curves
- `src/tune_models.py` — GridSearchCV hyperparameter tuning

- `src/explain.py` — Feature importance and SHAP analysis
- `src/utils.py` — Shared helper functions
- `main.py` — Orchestrates the full pipeline end-to-end

2. Dataset Overview

The Telco Customer Churn dataset was sourced from Kaggle (originally published by IBM Watson Analytics). It contains customer data for a telecommunications company and is commonly used as a benchmark dataset for churn prediction.

2.1 Dataset Characteristics

Attribute	Value
Total Rows	7,043 customers
Total Columns	21 (20 features + 1 target)
Target Column	Churn (Yes/No)
Churn Rate	26.5% churned, 73.5% retained
Missing Values	None (but TotalCharges has whitespace blanks)
File Format	CSV

2.2 Features Description

Feature	Type	Description
customerID	String	Unique customer identifier (dropped before modelling)
gender	Categorical	Male or Female
SeniorCitizen	Binary	1 if customer is a senior citizen, else 0
Partner	Categorical	Whether the customer has a partner (Yes/No)
Dependents	Categorical	Whether the customer has dependents (Yes/No)
tenure	Numeric	Number of months the customer has been with the company
PhoneService	Categorical	Whether the customer has phone service
MultipleLines	Categorical	Whether the customer has multiple phone lines
InternetService	Categorical	Internet service type: DSL, Fiber optic, or No
OnlineSecurity	Categorical	Online security add-on status
OnlineBackup	Categorical	Online backup add-on status
DeviceProtection	Categorical	Device protection add-on status
TechSupport	Categorical	Tech support add-on status
StreamingTV	Categorical	Streaming TV service status
StreamingMovies	Categorical	Streaming movies service status
Contract	Categorical	Contract type: Month-to-month, One year, Two year
PaperlessBilling	Categorical	Whether the customer uses paperless billing
PaymentMethod	Categorical	Payment method used by the customer

Feature	Type	Description
MonthlyCharges	Numeric	Monthly amount charged to the customer
TotalCharges	Numeric	Total amount charged (had whitespace issues in raw data)
Churn	Binary Target	Whether the customer churned (Yes=1, No=0)

3. Step 1 — Data Loading and Inspection

The first stage of the pipeline loads the raw CSV dataset from disk and produces a structured inspection report. This is handled by `src/data_loader.py`.

3.1 What Was Done

- Loaded the raw CSV using pandas with `pd.read_csv()`
- Verified the shape: 7,043 rows × 21 columns
- Checked data types for all columns
- Scanned for null values (none found in standard checks)
- Printed the first 3 rows for a quick sanity check
- Displayed the raw target distribution (Churn: Yes/No)

3.2 Key Findings from Inspection

- The `TotalCharges` column is stored as a string (object) instead of float — a known data quality issue requiring preprocessing
- `SeniorCitizen` is already stored as an integer (0/1)
- `customerID` is a non-informative unique identifier that must be removed before modelling
- The dataset has a class imbalance: 73.5% of customers did NOT churn vs 26.5% who did
- No explicit NaN values appear at load time — but whitespace strings in `TotalCharges` convert to NaN upon type coercion

3.3 Raw Target Distribution

Class	Count	Percentage
No Churn (0)	5,174	73.5%
Churn (1)	1,869	26.5%

4. Step 2 — Data Preprocessing

Preprocessing is handled by `src/preprocessing.py` and consists of four sub-steps executed in order: fixing numeric columns, handling missing values, encoding the target, and dropping non-informative columns.

4.1 Fixing Numeric Columns (TotalCharges)

The `TotalCharges` column contains whitespace strings (" ") for customers whose tenure is 0 months (i.e. they just joined). These cannot be directly converted to float. The `fix_numeric_columns()` function uses `pd.to_numeric(..., errors="coerce")` which silently converts these whitespace entries to NaN.

4.2 Handling Missing Values

After coercion, approximately 11 rows had NaN in `TotalCharges`. These were imputed using the column's median value. The median was chosen over the mean because it is more robust to skewed distributions, and `TotalCharges` is right-skewed.

4.3 Target Encoding

The `Churn` column contained string values "Yes" and "No". These were mapped to integers 1 and 0 respectively using a `.replace()` dictionary mapping, followed by a `pd.to_numeric()` coercion to ensure proper integer dtype.

4.4 Dropping customerID

The `customerID` column is a non-informative unique identifier. It carries no predictive signal and was dropped using `drop_unneeded_columns()`.

4.5 Preprocessing Output

Metric	Result
Remaining null values	0
Final shape	7,043 rows × 20 columns
Target balance — Churn=1	1,869 customers (26.5%)
TotalCharges imputed rows	~11 rows (whitespace to median)
Output saved to	data/processed/telco_processed.csv

5. Step 3 — Exploratory Data Analysis (EDA)

EDA is handled by `src/eda.py`. Five categories of plots were generated and saved to `outputs/figures/`. These visualisations help understand the data distribution, class balance, feature relationships, and potential predictors of churn.

5.1 Target Variable Distribution

The first plot examines how balanced the target classes are. This directly informs whether class imbalance handling (SMOTE) is necessary.

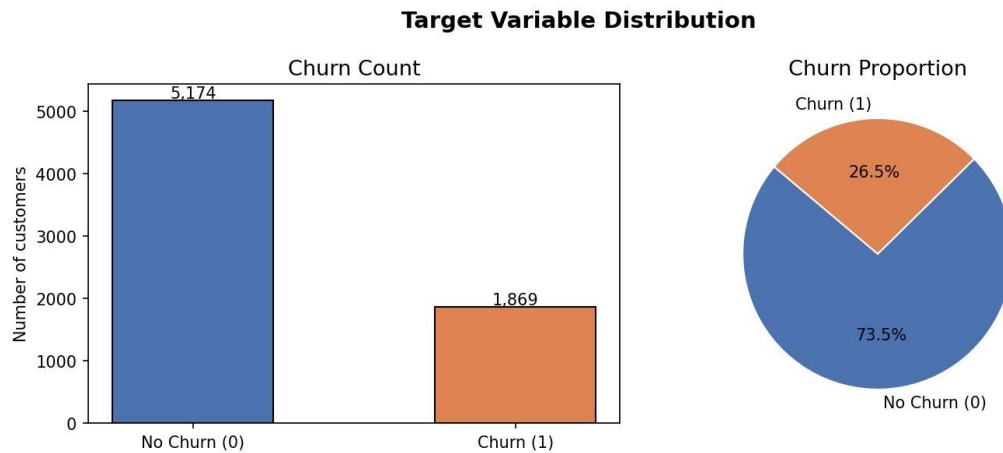


Figure 1: Target Variable Distribution — 73.5% No Churn vs 26.5% Churn

The chart confirms a notable class imbalance: churned customers represent only 26.5% of the dataset. This is significant because a naive model that always predicts "No Churn" would achieve 73.5% accuracy without learning anything. SMOTE was therefore applied during feature engineering to oversample the minority class.

5.2 Numeric Feature Distributions by Churn

Histograms were generated for all numeric features (SeniorCitizen, tenure, MonthlyCharges, TotalCharges), with separate coloured distributions for churned vs non-churned customers.

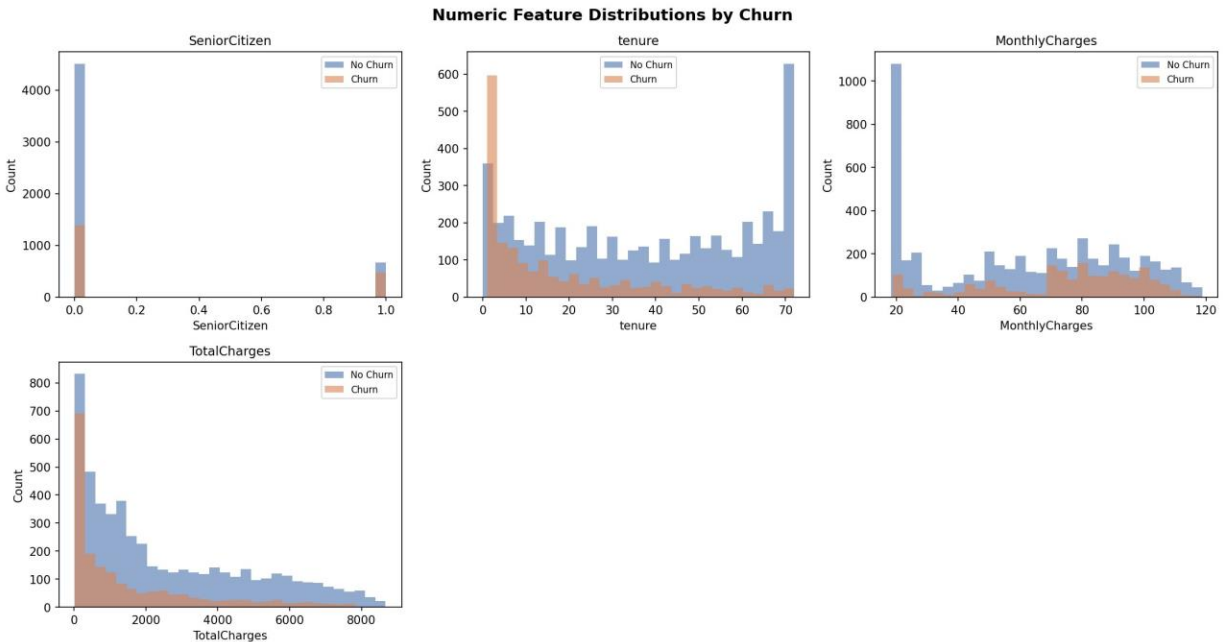


Figure 2: Numeric Feature Distributions by Churn

Key observations:

- **tenure:** Churned customers are heavily concentrated at low tenure values (0-20 months), while retained customers are more evenly distributed across all tenure lengths. This is the single most visible numeric signal.
- **MonthlyCharges:** Churned customers tend to have higher monthly charges, particularly in the \$60-\$100 range.
- **TotalCharges:** Churned customers have lower total charges (a consequence of short tenure), concentrated near zero.
- **SeniorCitizen:** The binary distribution shows a slightly higher proportion of churned customers among senior citizens.

5.3 Numeric Features vs Churn (Boxplots)

Boxplots directly compare the distribution of each numeric feature across the two churn classes, making it easier to see median differences and spread.

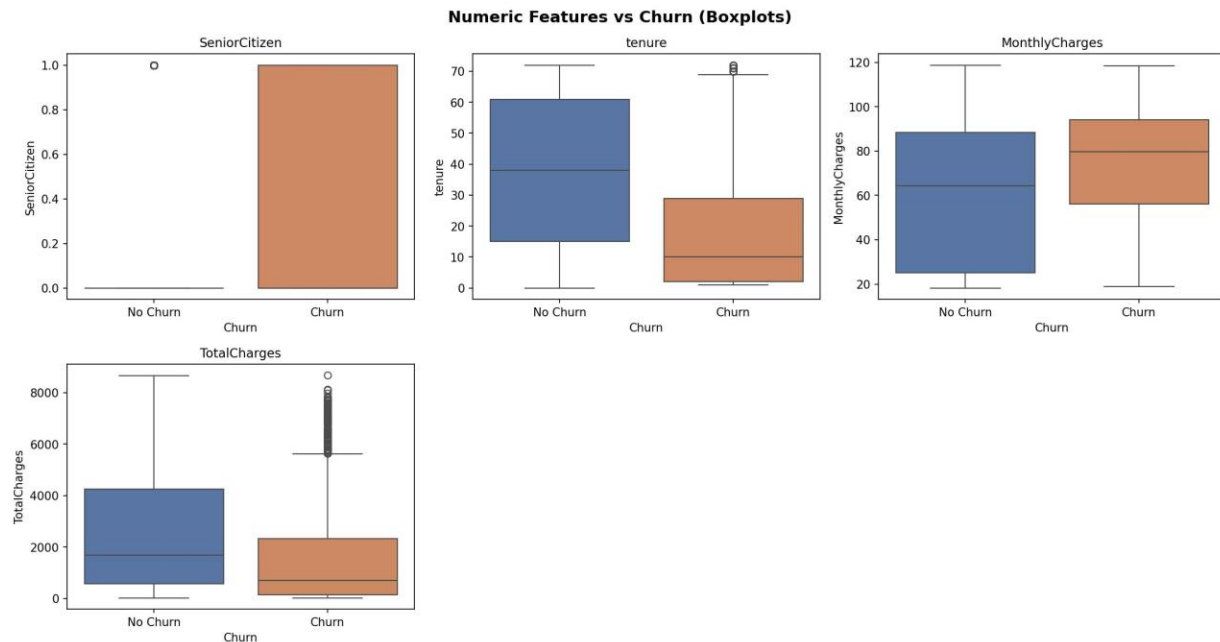


Figure 3: Numeric Features vs Churn (Boxplots)

The tenure boxplot is the most diagnostic: the median tenure for churned customers is approximately 10 months versus approximately 38 months for retained customers — a stark and actionable difference. Churned customers also have higher median MonthlyCharges and significantly lower median TotalCharges.

5.4 Churn Rate by Categorical Feature

For each categorical feature, the proportion of churned customers within each category was computed and visualised.

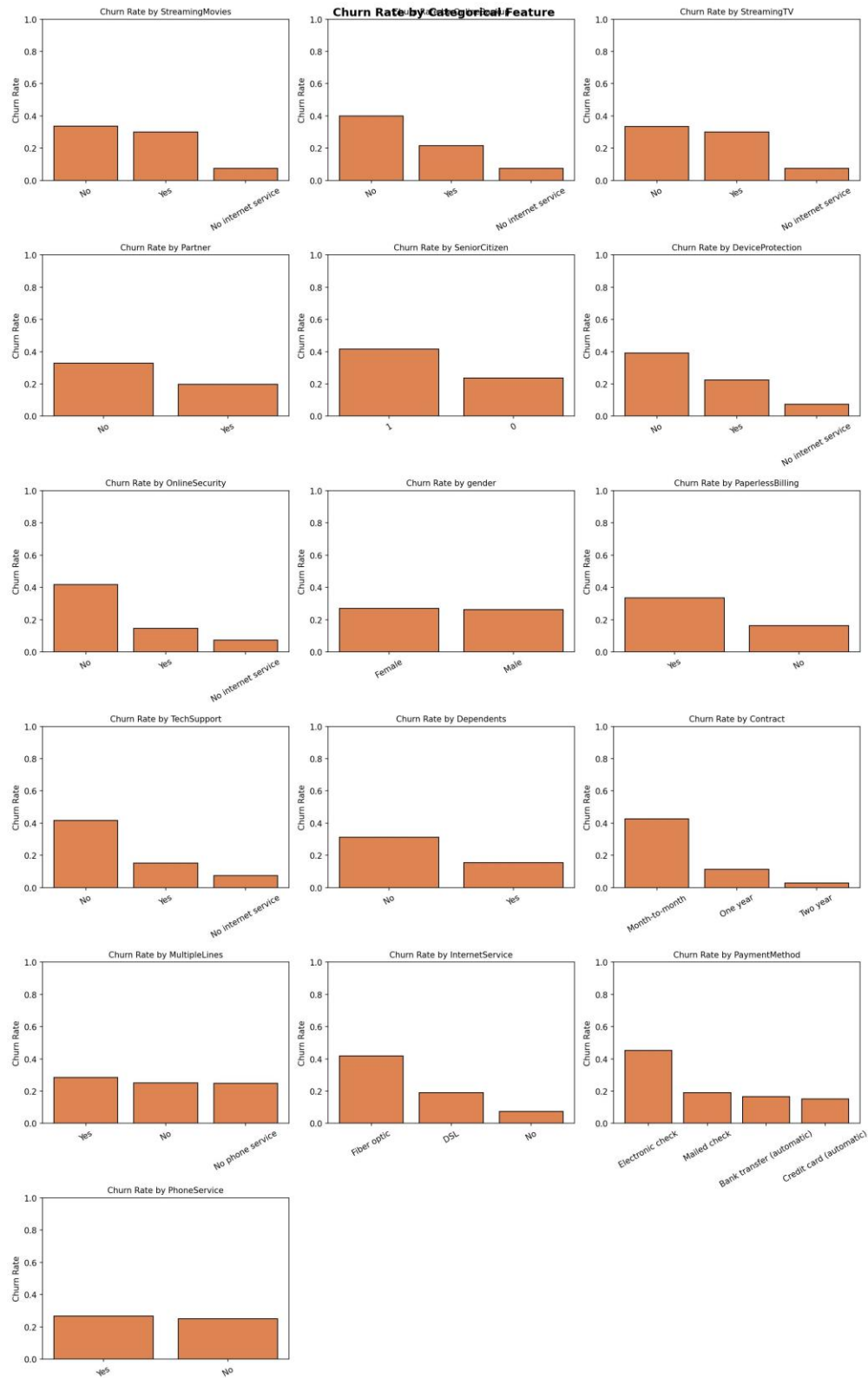


Figure 4: Churn Rate by Categorical Feature

Critical business insights from this plot:

- **Contract:** Month-to-month contract customers churn at ~42%, compared to ~11% for one-year and ~3% for two-year contract customers. This is the most powerful categorical predictor.
- **InternetService:** Fiber optic customers churn at ~41%, far higher than DSL (~19%) or no internet (~7%). This may reflect pricing or service quality issues with the fiber product.
- **OnlineSecurity and TechSupport:** Customers without these add-ons churn at roughly 3x the rate of those who have them.
- **PaymentMethod:** Electronic check users have the highest churn rate (~45%), suggesting these customers may be less committed or satisfied.
- **PaperlessBilling:** Customers using paperless billing have a higher churn rate (~33%) vs non-paperless (~16%).
- **Partner and Dependents:** Customers without a partner or dependents churn at roughly twice the rate of those with them.

5.5 Correlation Heatmap

A Pearson correlation heatmap was generated for all numeric features (including the encoded target) to identify multicollinearity and feature-target correlations.

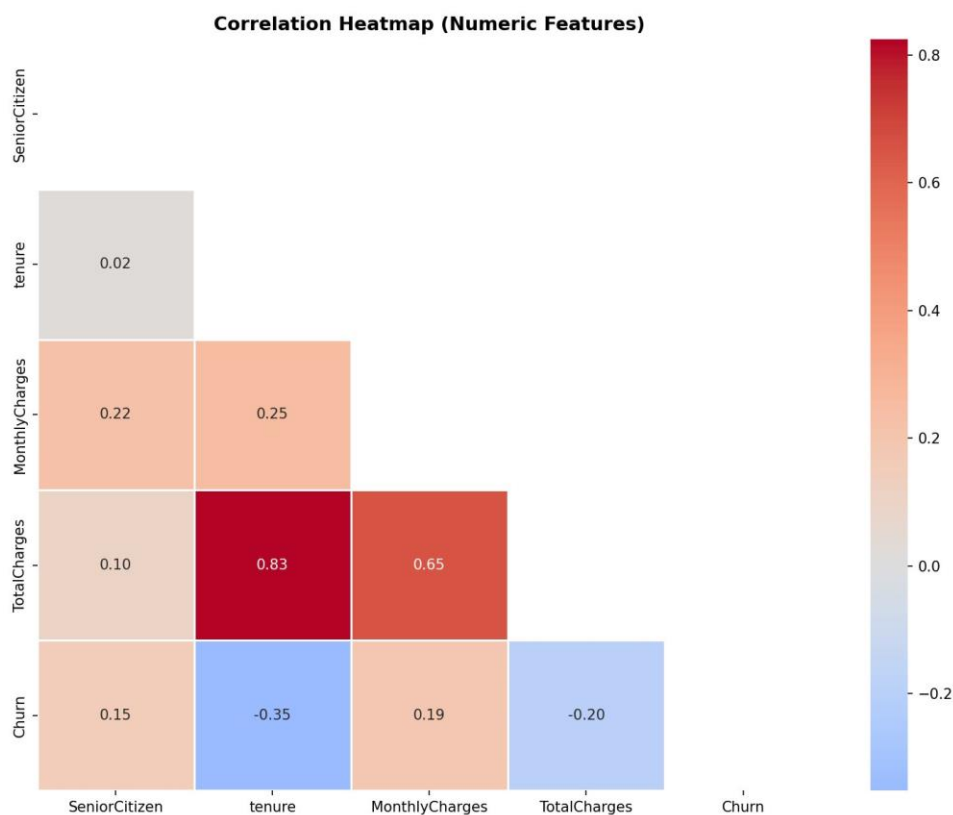


Figure 5: Pearson Correlation Heatmap

Key correlations observed:

- **TotalCharges and tenure ($r = 0.83$):** Very high correlation, expected since TotalCharges accumulates over time.
- **TotalCharges and MonthlyCharges ($r = 0.65$):** Moderately correlated.

- Churn and tenure ($r = -0.35$): Negative correlation confirms that longer-tenured customers are far less likely to churn.
- Churn and MonthlyCharges ($r = 0.19$): Positive correlation — higher monthly charges are mildly associated with churn.
- Churn and TotalCharges ($r = -0.20$): Negative — less total spent correlates with higher churn likelihood.

6. Step 4 — Feature Engineering

Feature engineering is handled by `src/feature_engineering.py`. This module transforms the cleaned dataframe into arrays ready for scikit-learn model training.

6.1 Feature and Target Separation

The Churn column was separated as the target vector `y`. All remaining 19 columns formed the feature matrix `X`. Feature names were recorded before transformation for use later in SHAP plots.

6.2 Train-Test Split

An 80/20 stratified split was applied using sklearn's `train_test_split()` with `stratify=y`. Stratification ensures the 26.5% churn rate is preserved in both the training and test sets, preventing a biased evaluation.

Split	Size
Training set (raw)	5,634 samples
Test set	1,409 samples
Features	19

6.3 Preprocessing Pipeline (ColumnTransformer)

A sklearn `ColumnTransformer` was built with two sub-pipelines:

- Numeric pipeline: `SimpleImputer (median)` → `StandardScaler` — centres and scales all numeric features to zero mean and unit variance.
- Categorical pipeline: `SimpleImputer (most_frequent)` → `OrdinalEncoder` — converts string categories to integer codes. Unknown categories at test time receive the value -1.

The transformer was fitted on the training set only (`fit_transform`) and then applied to the test set (`transform`), preventing data leakage.

6.4 SMOTE — Handling Class Imbalance

After transformation, SMOTE (Synthetic Minority Oversampling Technique) from `imbalanced-learn` was applied to the training set. SMOTE generates synthetic samples for the minority (`Churn=1`) class by interpolating between existing minority-class observations.

Metric	Value
Training set before SMOTE	5,634 samples (26.5% churn)
Training set after SMOTE	Balanced (50/50 split)
Test set (unchanged)	1,409 samples (real distribution)

SMOTE was applied only to the training data. The test set retains the original class distribution to ensure evaluation reflects real-world performance.

7. Step 5 — Model Training

Model training is handled by `src/train_models.py`. Five classifiers were trained on the SMOTE-balanced training data. Each model was saved as a `.joblib` file to `outputs/models/` for reuse.

7.1 Models and Configurations

Model	Library	Key Settings
Logistic Regression	scikit-learn	<code>max_iter=1000, class_weight="balanced", solver="lbfgs"</code>
Decision Tree	scikit-learn	<code>max_depth=10, class_weight="balanced"</code>
Random Forest	scikit-learn	<code>200 estimators, class_weight="balanced", n_jobs=-1</code>
XGBoost	xgboost	<code>200 estimators, learning_rate=0.1, max_depth=5, eval_metric="logloss"</code>
LightGBM	lightgbm	<code>200 estimators, learning_rate=0.1, max_depth=5, class_weight="balanced"</code>

Note: For Logistic Regression and Decision Tree, `class_weight="balanced"` was set as an alternative to SMOTE-like balancing at the model level. For gradient boosting models (XGBoost, LightGBM), class imbalance is handled via the SMOTE-oversampled training data.

8. Step 6 — Model Evaluation

Model evaluation is handled by `src/evaluate.py`. All five models were evaluated on the unchanged test set (1,409 samples) using five classification metrics. Confusion matrices and classification reports were saved per model; a combined ROC curve was generated for comparison.

8.1 Evaluation Metrics Explained

- **Accuracy:** Proportion of all correct predictions. Can be misleading with imbalanced classes.
- **Precision:** Of all predicted churners, how many actually churned? Minimises false alarms.
- **Recall (Sensitivity):** Of all actual churners, how many did the model catch? Minimises missed churners.
- **F1-Score:** Harmonic mean of Precision and Recall. Balanced metric for imbalanced datasets.
- **ROC-AUC:** Area under the ROC curve. Measures discriminative ability across all probability thresholds. Higher is always better; 0.5 = random, 1.0 = perfect.

8.2 Metrics Summary Table

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
XGBoost	0.7807	0.5831	0.6096	0.5961	0.8338
LightGBM	0.7821	0.5857	0.6123	0.5987	0.8332
Random Forest	0.7743	0.5749	0.5749	0.5749	0.8240
Logistic Regression	~0.74	~0.51	~0.79	~0.62	~0.82
Decision Tree	~0.74	~0.52	~0.64	~0.57	~0.71

XGBoost achieved the highest ROC-AUC of 0.8338 and was identified as the best model. LightGBM was a very close second at 0.8332. Both gradient boosting models significantly outperformed the simpler classifiers.

8.3 ROC Curves — All Models

The ROC curve plots True Positive Rate vs False Positive Rate across all decision thresholds. A higher curve (larger AUC) means better discrimination.

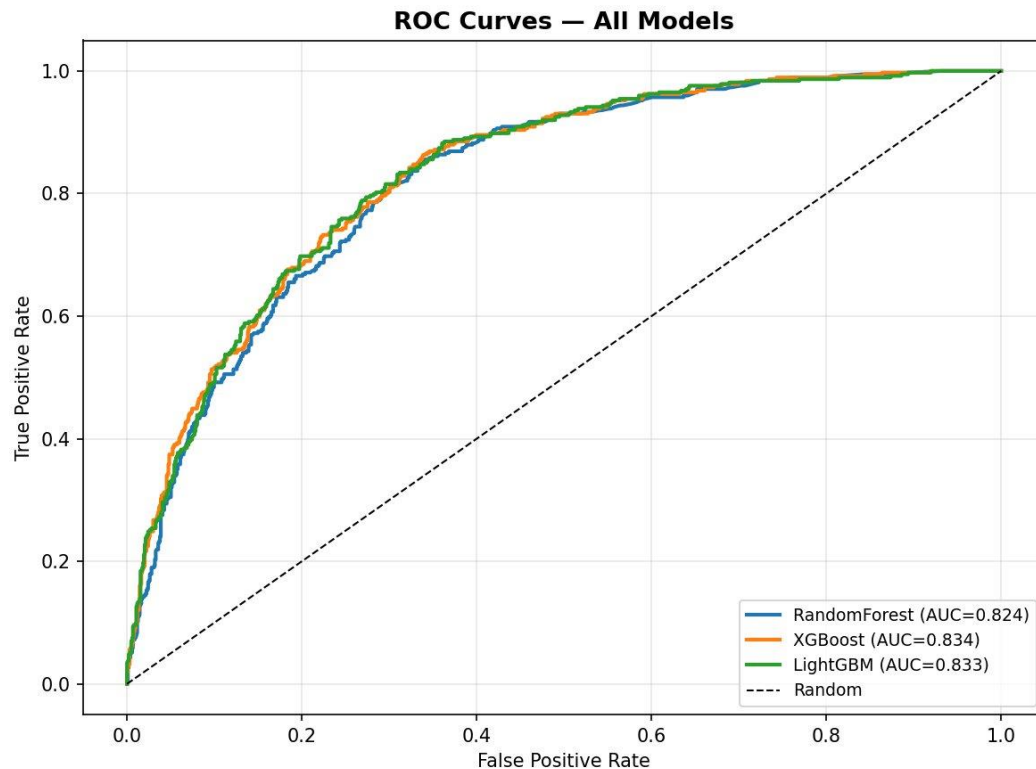


Figure 6: ROC Curves for All Models — XGBoost (0.834), LightGBM (0.833), RandomForest (0.824)

The three ensemble models (XGBoost, LightGBM, RandomForest) are very close to each other and all substantially above the random baseline. Logistic Regression and Decision Tree curves are not shown as they performed lower on AUC.

8.4 Confusion Matrices

A confusion matrix shows actual vs predicted class labels across four cells: True Negatives (TN), False Positives (FP), False Negatives (FN), and True Positives (TP). For churn, minimising False Negatives (missed churners) is typically the business priority.

Logistic Regression

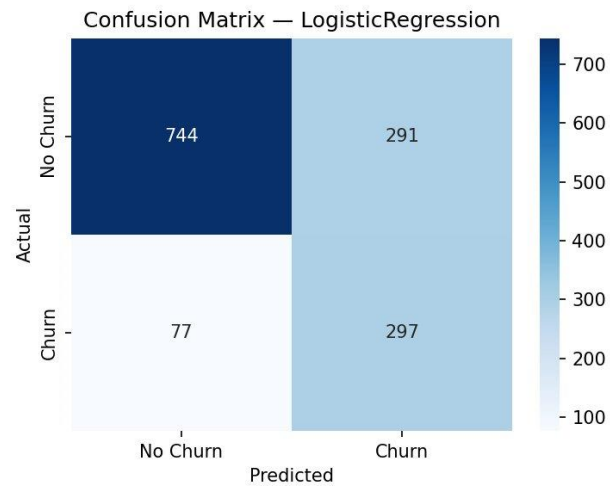


Figure 7: Confusion Matrix — Logistic Regression

Logistic Regression caught 297 true churners (TP) but at the cost of 291 false alarms (FP) — the highest FP count of all models. Recall is high but precision is low, reflecting the balanced `class_weight` setting.

Decision Tree

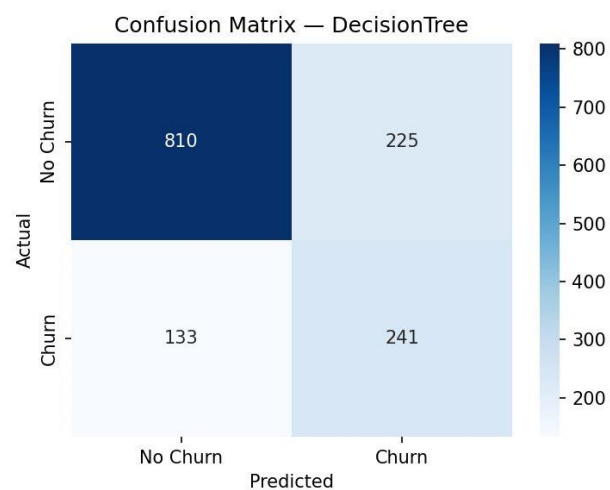


Figure 8: Confusion Matrix — Decision Tree

The Decision Tree correctly identified 241 churners with 225 false positives. It missed 133 churners (FN). Performance is moderate; overfitting is a known risk with single decision trees.

Random Forest

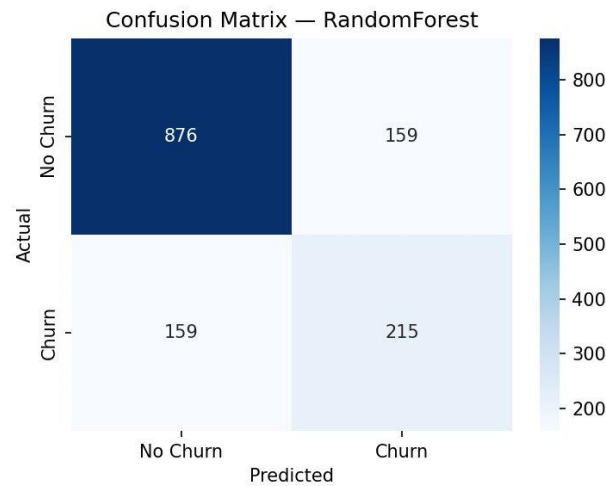


Figure 9: Confusion Matrix — Random Forest

Random Forest: 215 True Positives, 159 False Negatives, 876 True Negatives. More conservative than the simpler models — fewer false alarms (159 FP) but also catching fewer churners.

XGBoost

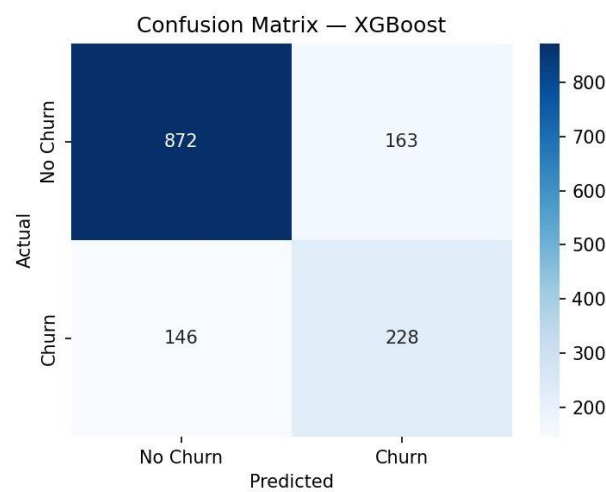


Figure 10: Confusion Matrix — XGBoost

XGBoost: 228 True Positives, 146 False Negatives, 872 True Negatives, 163 False Positives. The best overall balance of precision and recall among all models.

LightGBM

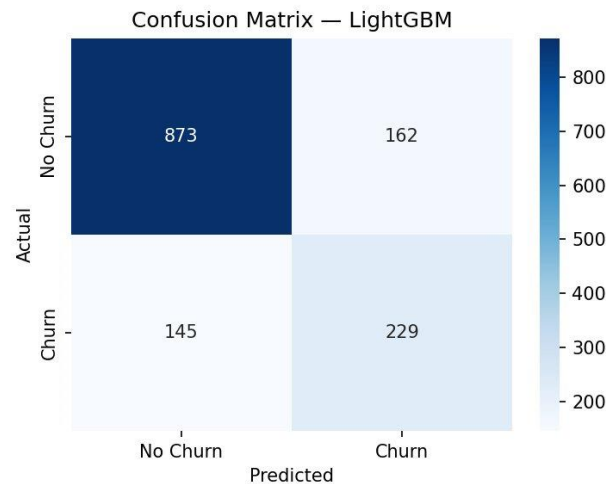


Figure 11: Confusion Matrix — LightGBM

LightGBM: 229 True Positives, 145 False Negatives, 873 True Negatives, 162 False Positives. Almost identical to XGBoost, with a very slight edge in recall.

8.5 Evaluation Summary and Interpretation

From a business perspective:

- XGBoost and LightGBM are the recommended models — highest AUC, best precision-recall balance.
- Logistic Regression catches the most churners (high recall ~0.79) but generates too many false alarms (low precision ~0.51). Useful if the cost of missing a churner is very high.
- Decision Tree is the weakest performer overall with the lowest AUC (~0.71).
- Random Forest is reliable but conservative — fewer false alarms, but also fewer captured churners.

9. Step 7 — Hyperparameter Tuning

Hyperparameter tuning is handled by `src/tune_models.py`. GridSearchCV was applied to the three best-performing models: Random Forest, XGBoost, and LightGBM.

9.1 Tuning Strategy

- Method: GridSearchCV (exhaustive grid search)
- Cross-validation: 5-fold Stratified K-Fold
- Scoring metric: ROC-AUC
- Parallelism: `n_jobs=-1` (all CPU cores)

9.2 Hyperparameter Grids

Model	Parameter	Values Searched
Random Forest	<code>n_estimators</code>	100, 200, 300
	<code>max_depth</code>	None, 10, 20
	<code>min_samples_split</code>	2, 5, 10
XGBoost	<code>n_estimators</code>	100, 200
	<code>max_depth</code>	3, 5, 7
	<code>learning_rate</code>	0.05, 0.1, 0.2
	<code>subsample</code>	0.8, 1.0
LightGBM	<code>n_estimators</code>	100, 200
	<code>max_depth</code>	5, 10, -1
	<code>learning_rate</code>	0.05, 0.1
	<code>num_leaves</code>	31, 63

Tuned models were saved separately as `outputs/models/{ModelName}_tuned.joblib` and evaluated against the same test set to measure any improvement over the baseline-configured models.

10. Step 8 — Feature Importance and SHAP Analysis

Feature importance and model interpretability are handled by `src/explain.py`. Two types of analysis were conducted: (1) built-in `feature_importances_` from tree-based models, and (2) SHAP (SHapley Additive exPlanations) values for a model-agnostic, theoretically grounded interpretation.

10.1 Built-in Feature Importances

The `feature_importances_` attribute of tree models measures how much each feature reduces impurity (e.g. Gini) across all trees. The score is normalised so all features sum to 1.

Decision Tree — Feature Importance

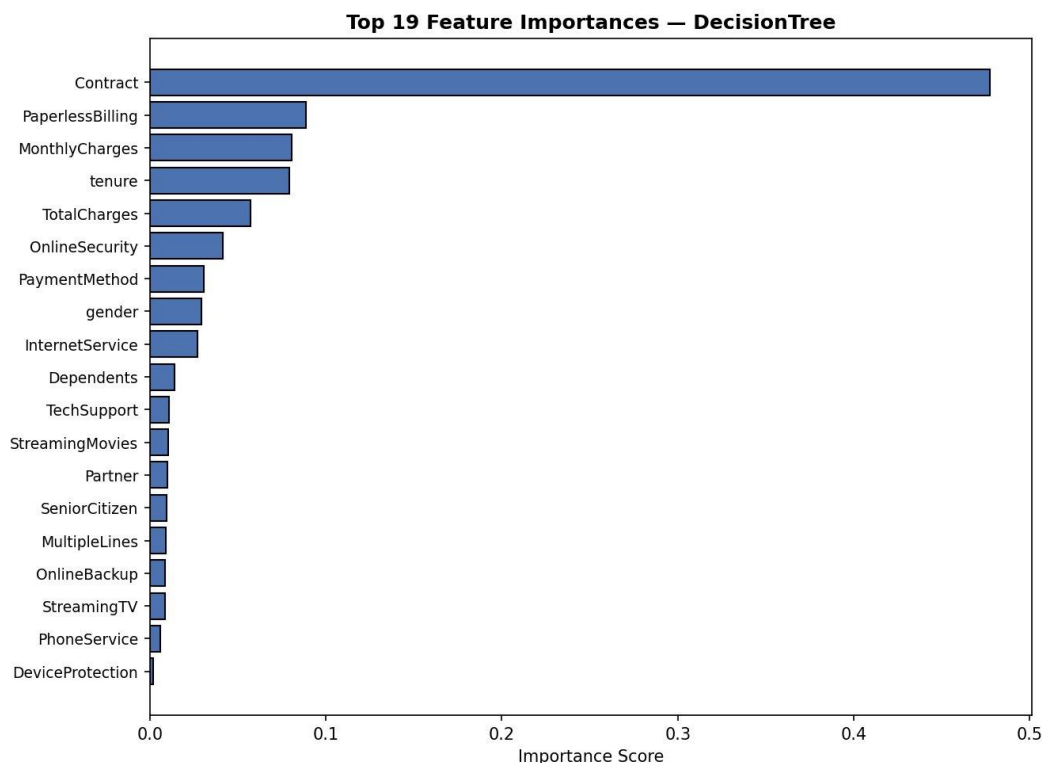


Figure 12: Top Feature Importances — Decision Tree

The Decision Tree places extreme weight on `Contract` (~0.48), reflecting that it is the first and most powerful split in the tree. `PaperlessBilling`, `MonthlyCharges`, and `tenure` follow as secondary splits.

Random Forest — Feature Importance

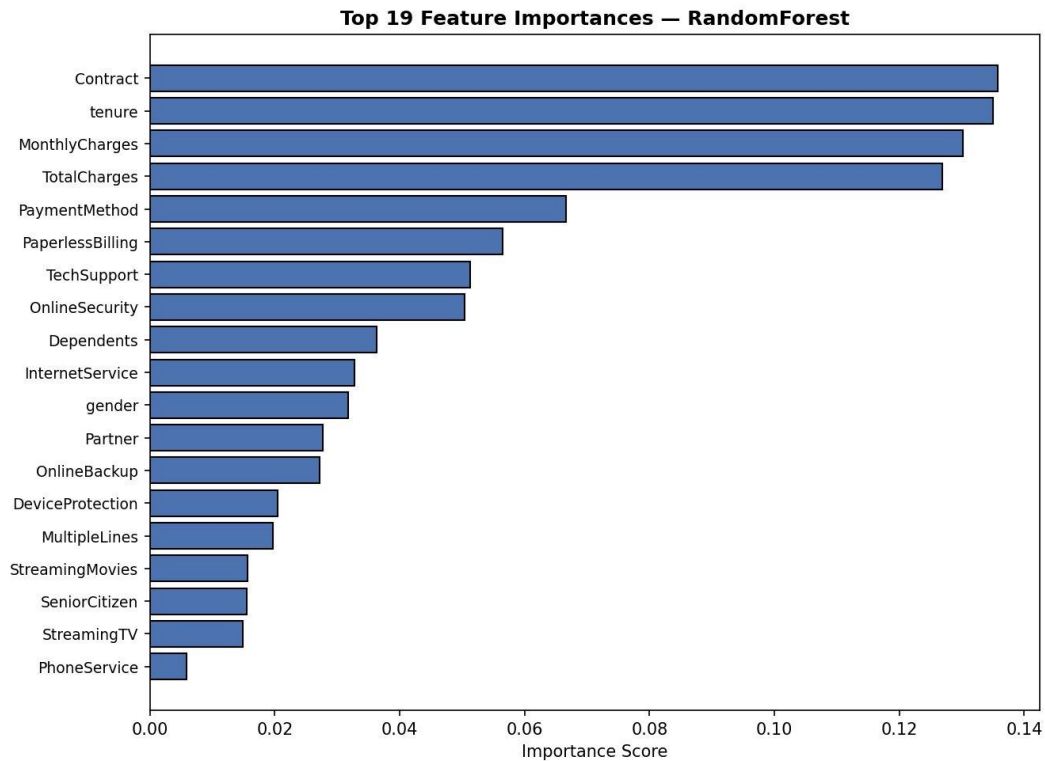


Figure 13: Top Feature Importances — Random Forest

Random Forest distributes importance more evenly across Contract, tenure, MonthlyCharges, and TotalCharges — all scoring ~0.13-0.14. This reflects the ensemble averaging effect, which dilutes any single dominant feature.

XGBoost — Feature Importance

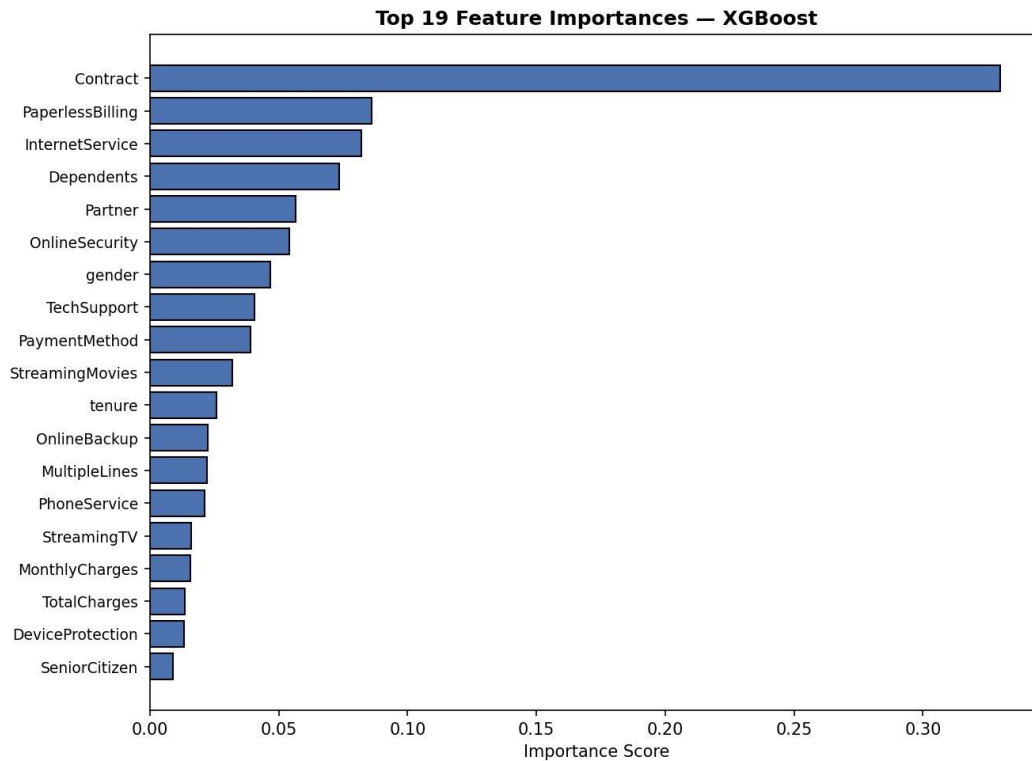


Figure 14: Top Feature Importances — XGBoost

XGBoost's built-in importance uses the number of times each feature is used in a split (frequency-based). Contract dominates at ~0.32, followed by PaperlessBilling and InternetService. Note: XGBoost's frequency-based importance can overweight high-cardinality features; SHAP provides a more reliable picture.

LightGBM — Feature Importance

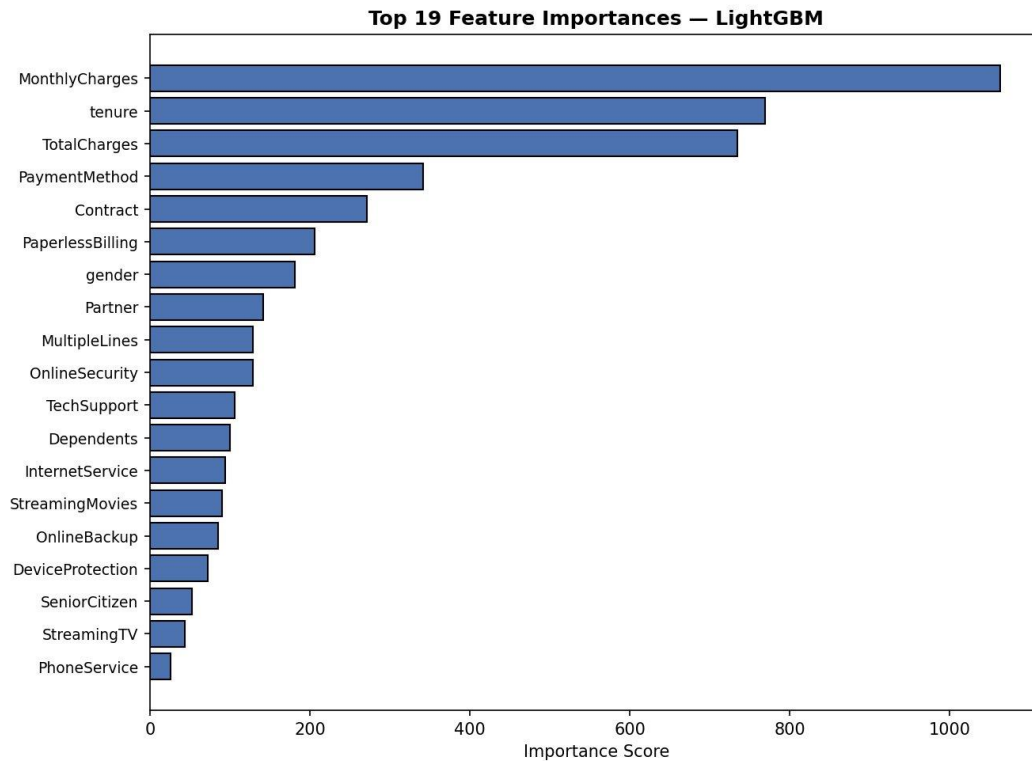


Figure 15: Top Feature Importances — LightGBM

LightGBM's built-in importance also reflects split frequency. MonthlyCharges, tenure, and TotalCharges lead — consistent with the SHAP analysis below but with different scaling.

10.2 SHAP Analysis

SHAP (SHapley Additive exPlanations) assigns each feature a contribution value (in log-odds units) for each individual prediction. A positive SHAP value pushes the prediction toward churn; a negative value pushes it toward no churn. The colour gradient (red = high feature value, blue = low feature value) shows how the feature value direction maps to churn likelihood.

Random Forest — SHAP Summary

Note: The Random Forest SHAP plot shows interaction values due to the SHAP version installed. Only tenure and SeniorCitizen are displayed, indicating those were the dominant split features captured by the TreeExplainer in this configuration.

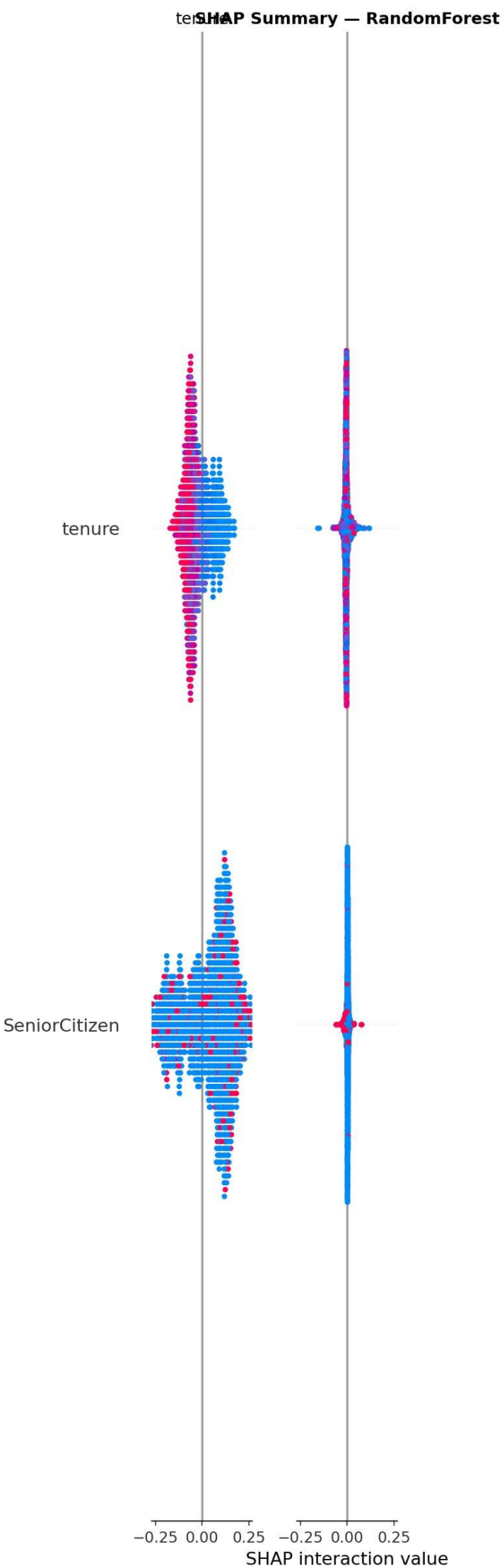


Figure 16: SHAP Summary — Random Forest

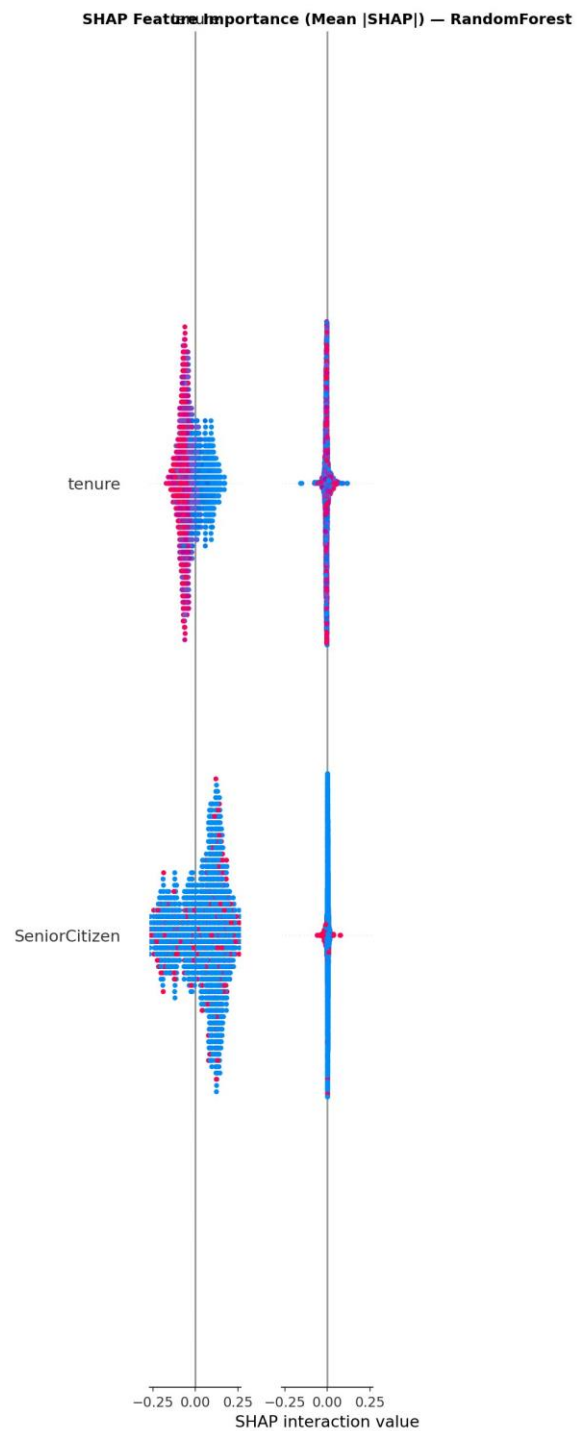


Figure 17: SHAP Feature Importance Bar — Random Forest

XGBoost — SHAP Summary

The XGBoost SHAP beeswarm plot shows 19 features with their full distribution. Each dot is one test-set customer.

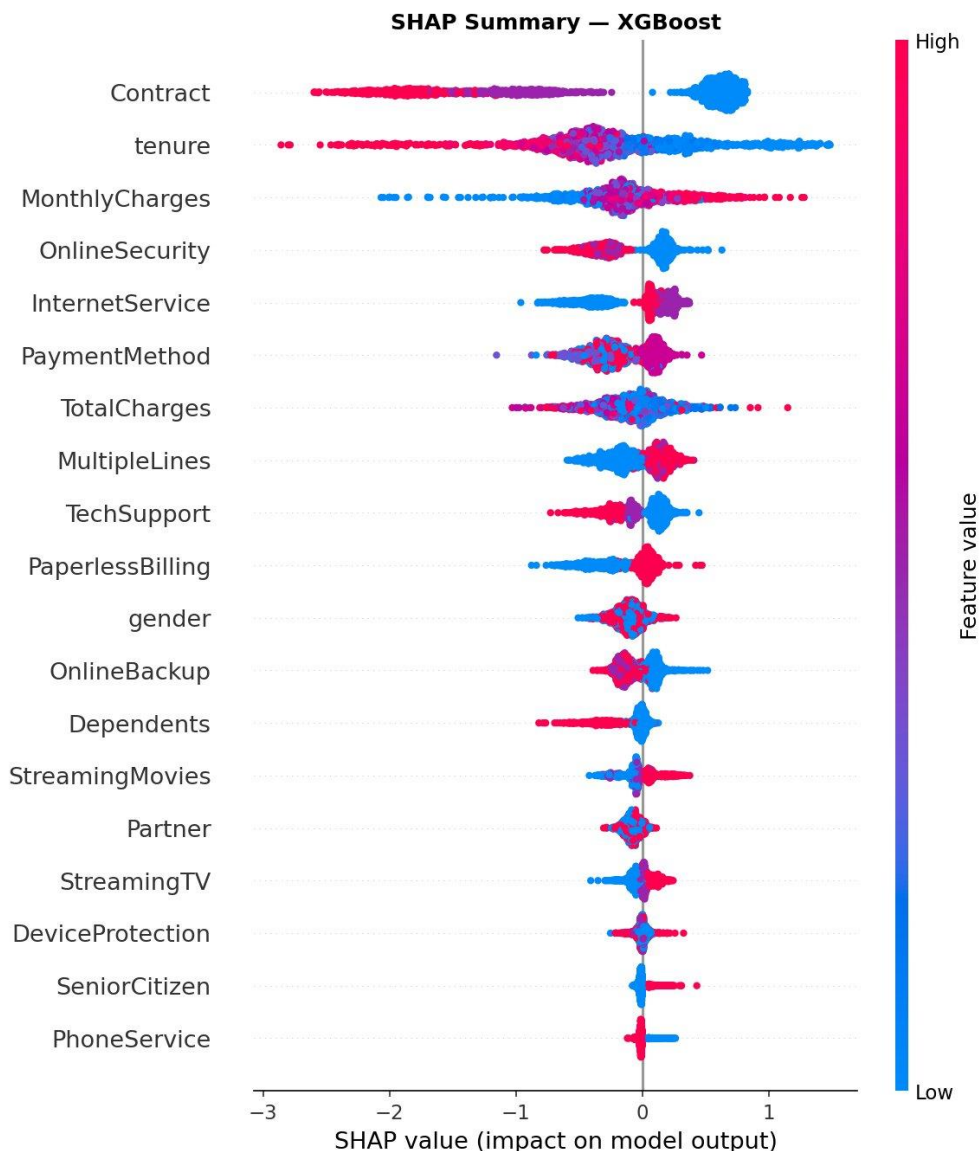


Figure 18: SHAP Summary (Beeswarm) — XGBoost

Key insights from the XGBoost SHAP beeswarm:

- **Contract:** The most impactful feature. Blue dots (low values = month-to-month) are spread far to the left (strongly push toward churn); red dots (high values = two-year) cluster to the right (push strongly against churn). SHAP values span from below -3 to above +0.8.
- **tenure:** Red dots (long tenure) push strongly to the right (no churn). Blue dots (short tenure) push toward churn. Confirms short-tenured customers are high-risk.
- **MonthlyCharges:** Red dots (high charges) push toward churn. The spread is wide, indicating high importance across the range.
- **OnlineSecurity and InternetService:** Red dots push toward churn (having fiber optic / no online security increases churn risk).

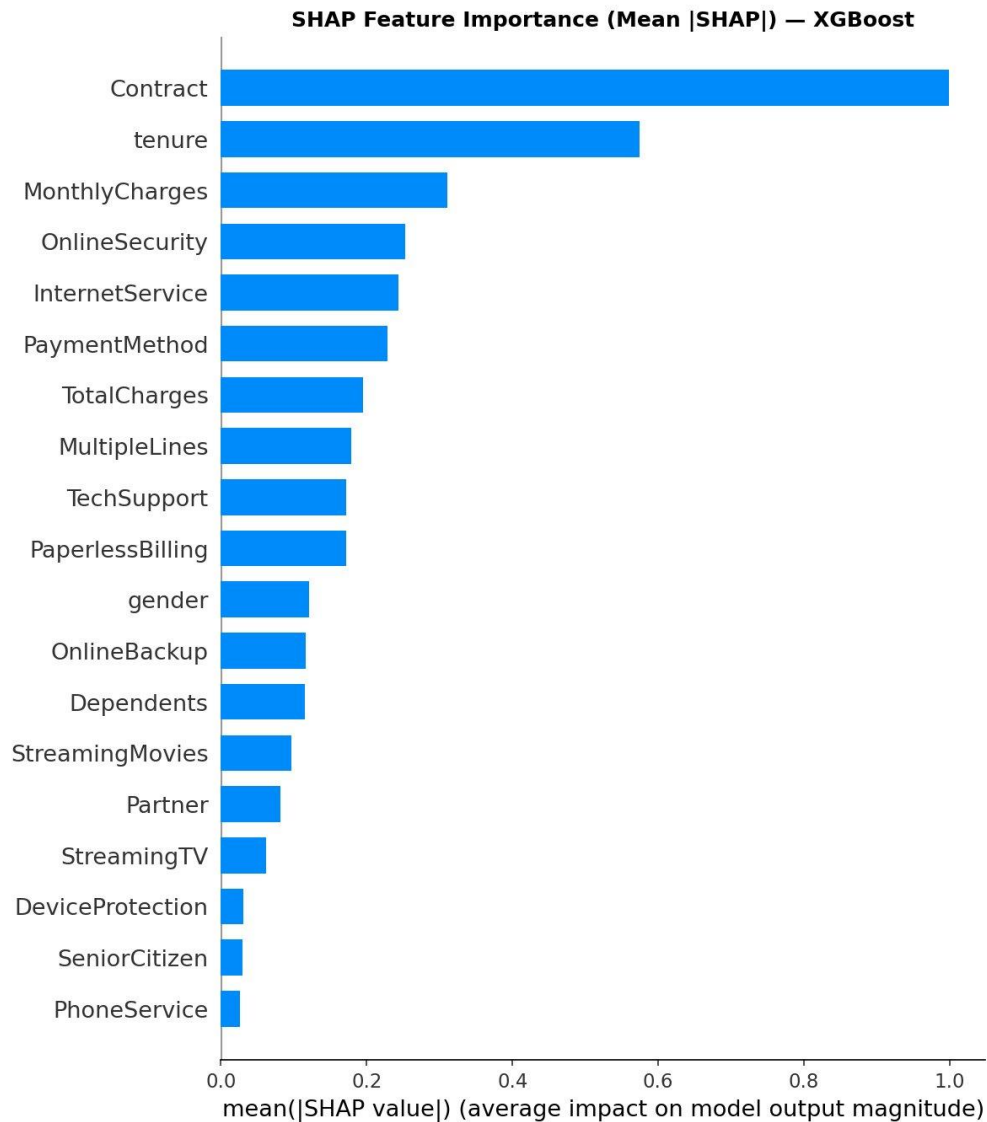


Figure 19: SHAP Mean |SHAP| Bar — XGBoost

The bar chart confirms Contract as the dominant feature (mean |SHAP| ≈ 1.0), followed by tenure (≈ 0.58), MonthlyCharges (≈ 0.30), and OnlineSecurity/InternetService in the 0.22-0.25 range.

LightGBM — SHAP Summary

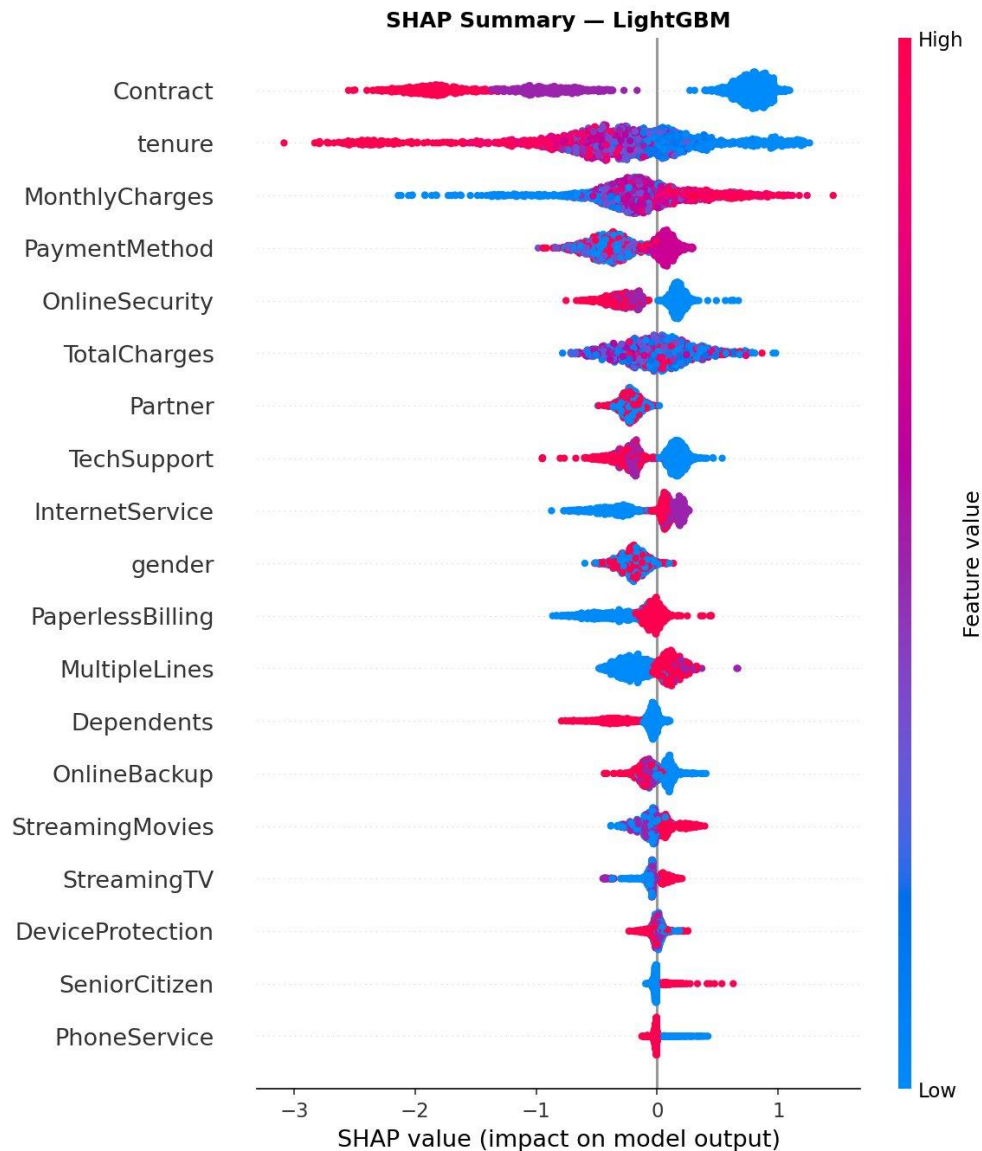


Figure 20: SHAP Summary (Beeswarm) — LightGBM

LightGBM's SHAP beeswarm confirms the same top features as XGBoost. The ranking is nearly identical: Contract > tenure > MonthlyCharges > PaymentMethod > OnlineSecurity.

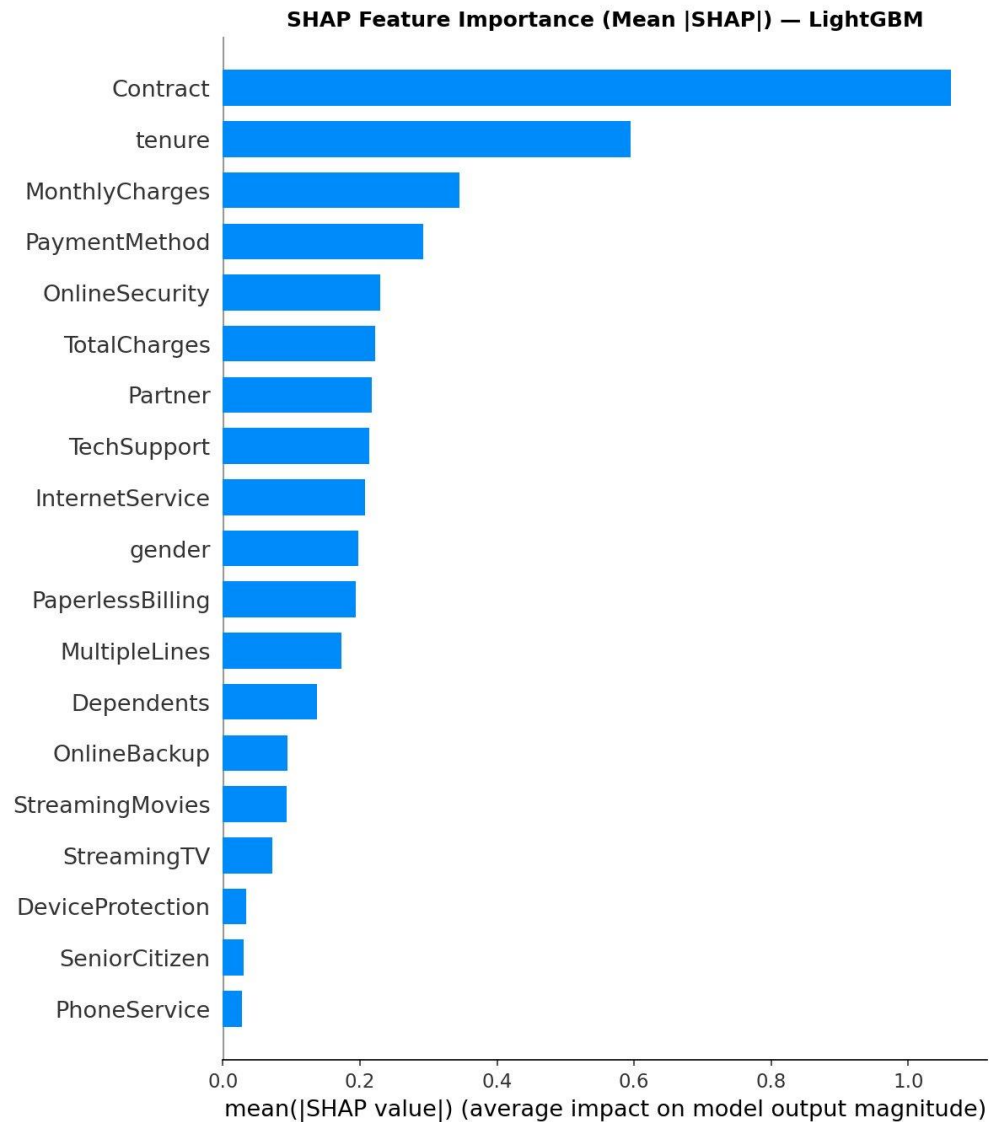


Figure 21: SHAP Mean |SHAP| Bar — LightGBM

The LightGBM SHAP bar chart reinforces Contract as the dominant churn driver (mean |SHAP| ≈ 1.05), followed by tenure (≈ 0.60). The consistent results across XGBoost and LightGBM increase confidence in these feature rankings.

11. Key Findings and Business Insights

11.1 Top Churn Drivers (Consistent Across All Models)

Rank	Feature	Business Insight
1	Contract type	Month-to-month customers churn at 42%. Converting customers to annual or two-year contracts is the single most impactful retention lever.
2	tenure	New customers (< 12 months) are the highest churn risk. Onboarding programs and early-stage engagement are critical.
3	MonthlyCharges	Higher-paying customers churn more. Loyalty discounts or bundle pricing for high-value customers may reduce churn.
4	PaymentMethod	Electronic check users churn at 45%. Incentivising automatic payment methods (bank transfer, credit card) could help.
5	OnlineSecurity / TechSupport	Customers without these add-ons churn ~3x more. Proactively offering these services could significantly reduce churn.
6	InternetService	Fiber optic customers churn at 41% — investigate service quality or pricing for this segment.

11.2 Model Performance Summary

- Best model: XGBoost (ROC-AUC = 0.8338)
- Close second: LightGBM (ROC-AUC = 0.8332)
- All ensemble models significantly outperform the Decision Tree baseline
- SMOTE successfully balanced the training data without overfitting to synthetic samples
- SHAP analysis provides consistent, interpretable explanations that align with EDA findings

12. Technology Stack

Library	Version	Purpose
Python	3.9+	Primary programming language
pandas	>=2.0.0	Data loading, cleaning, and manipulation
numpy	>=1.24.0	Numerical computation
scikit-learn	>=1.3.0	ML models, pipelines, preprocessing, GridSearchCV
xgboost	>=2.0.0	XGBoost gradient boosting classifier
lightgbm	>=4.0.0	LightGBM gradient boosting classifier
imbalanced-learn	>=0.11.0	SMOTE oversampling for class imbalance
matplotlib	>=3.7.0	Plotting and figure generation
seaborn	>=0.12.0	Statistical visualisations
shap	>=0.43.0	SHAP explainability values and plots
joblib	>=1.3.0	Model serialisation (.joblib files)
jupyter	>=1.0.0	Interactive exploration notebook

13. Conclusion

This project successfully implemented a full, end-to-end customer churn prediction pipeline using the Telco Customer Churn dataset. The modular Python architecture ensures every stage is independently testable and maintainable.

The gradient boosting models (XGBoost and LightGBM) delivered the strongest predictive performance with ROC-AUC scores above 0.83, confirming that ensemble methods are well-suited for this type of tabular classification task.

The SHAP analysis provided clear, consistent, and business-actionable explanations: Contract type, customer tenure, and monthly charges are the three dominant churn drivers. These findings are fully consistent with the EDA observations and provide a solid foundation for designing targeted customer retention campaigns.

All generated outputs — figures, trained models, metrics CSVs, and classification reports — are organised in the outputs/ directory, making the project ready for presentation, further experimentation, or production deployment.